Front-End Web Technology
(2504CS305)

ReactJS Unit 4: Routing, Hooks, Forms and Consuming APIs

Complete Easy-to-Understand Exam Study Notes

Based on: unit-4.pptx

Table of Contents

1. Introduction and Syllabus Coverage
2. Routing in React
3. React CSS Styling
4. React Hooks
5. useState Hook
6. useEffect Hook
7. Forms in React
8. Controlled Components
9. Handling Multiple Inputs
10. Consuming APIs in ReactJS
11. Mock API using mockapi.io
12. CRUD Operations using React
13. Exam Practice Questions
14. Final Revision

1. Introduction

What is ReactJS?

ReactJS is a JavaScript library used to build user interfaces (UI). It is mainly used to create fast, interactive single-page web applications.

What is a Component?

A component is a small, reusable part of a web page. For example, a Header, Footer, Button, Form, or Faculty card can each be a component.

```
function Home() {
return <h1>Home Page</h1>;
}

export default Home;
```

Explanation:

• **function Home()** creates a component named Home.
• **return** returns JSX code, which looks like HTML.
• The h1 tag displays a heading.
• **export default Home** allows another file to import this component.

Unit 4 Syllabus

Topic	What you will learn
Routing	Show different components for different URLs
React CSS Styling	Apply styles using inline CSS, CSS files and CSS Modules
Hooks	Use React features in function components
useState	Store and update changing data
useEffect	Perform side effects such as API calls
Forms	Collect user input
Controlled Components	Connect form inputs to React state
Consuming API	Get data from a server using fetch()
CRUD Operations	Create, Read, Update and Delete API data

2. Routing in React

Simple Definition

Routing means displaying different components based on the URL of the browser.

Detailed Explanation

In a normal website, every URL may load a new HTML page. In React, routing changes the displayed component without completely reloading the page. This makes the application faster and smoother.

Example:
• URL **/** displays the Home component.
• URL **/about** displays the About component.
• URL **/contact** displays the Contact component.

Install React Router

React does not include routing by default. We must install the **react-router-dom** package.

```
npm install react-router-dom
```

Recommended Folder Structure

```
src/
pages/
Layout.js
Home.js
About.js
Contact.js
index.js
```

Important Routing Components

Component	Purpose
BrowserRouter	Enables routing in the React application
Routes	Contains all route definitions
Route	Connects one URL path to one component
Link	Creates a navigation link
Outlet	Displays the child route component
useParams	Reads values from the URL
useNavigate	Moves the user to another route programmatically

Routing Example: index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import {
BrowserRouter,
Routes,
Route
} from "react-router-dom";

import Layout from "./pages/Layout";
import Home from "./pages/Home";
import About from "./pages/About";
import Contact from "./pages/Contact";

const root = ReactDOM.createRoot(
document.getElementById("root")
```

```
);

root.render(
<BrowserRouter>
<Routes>
<Route path="/" element={<Layout />}>
<Route index element={<Home />} />
<Route path="about" element={<About />} />
<Route path="contact" element={<Contact />} />
</Route>
</Routes>
</BrowserRouter>
);
```

Line-by-Line Explanation

Code	Meaning
import React from 'react'	Imports React
import ReactDOM from 'react-dom/client'	Used to render React application in browser
import { BrowserRouter, Routes, Route }	Imports routing components
import Layout from './pages/Layout'	Imports Layout component
const root = ReactDOM.createRoot(...)	Finds HTML element with id root
root.render(...)	Displays React application
BrowserRouter	Starts routing support
Routes	Contains route definitions
Route path='/' element={Layout}	For URL starting with /, display Layout
Route index element={Home}	Default child route is Home
Route path='about'	URL /about displays About
Route path='contact'	URL /contact displays Contact

2.1 Layout, Link and Outlet

Layout.js

```
import { Link, Outlet } from "react-router-dom";

function Layout() {
return (
<>
<h1>Header</h1>

<ul>
<li><Link to="/">Home</Link></li>
<li><Link to="/about">About</Link></li>
<li><Link to="/contact">Contact</Link></li>
</ul>

<Outlet />

<h1>Footer</h1>
</>
);
}

export default Layout;
```

Explanation

- **Link to='/'**: Creates a link to the Home page.
- **Link to='/about'**: Creates a link to the About page.
- **Outlet**: Displays the selected child component.
- **React Fragment**: Groups JSX elements without adding an extra HTML element.

Home.js

```
function Home() {
return <h1>Home Page</h1>;
}

export default Home;
```

About.js

```
function About() {
return <h1>About Page</h1>;
}

export default About;
```

Contact.js

```
function Contact() {
return <h1>Contact Page</h1>;
}

export default Contact;
```

Routing Flow

```
Browser URL: /about
|
v
BrowserRouter checks routes
|
v
```

```
Layout is displayed
Header + Menu + Footer
|
v
Outlet displays About component
|
v
Output: Header, Menu, About Page, Footer
```

Important Exam Points

- React does not provide routing by default.
- The routing package is react-router-dom.
- BrowserRouter enables routing.
- Routes contains multiple Route components.
- Link is used instead of normal HTML anchor tag.
- Outlet displays child route components.

3. React CSS Styling

Simple Definition

CSS styling is used to change the appearance of React components, such as color, background, font size, margin and padding.

Three Common Methods

Method	Where CSS is written	Best use
Inline Styling	Directly inside JSX	Small, component-specific style
CSS Stylesheet	Separate .css file	Common global styles
CSS Module	Separate .module.css file	Component-specific styles without name conflicts

Method 1: Inline Styling

In inline styling, CSS is written directly in the style attribute. The value must be a JavaScript object.

```
function Home() {
return (
<h1
style={{
color: "red",
backgroundColor: "blue"
}}
>
Hello World
</h1>
);
}
```

Line-by-Line Explanation

- **style={{ ... }}**: Outer curly brackets mean JavaScript expression; inner curly brackets create a JavaScript object.
- **color: 'red'**: Makes text red.
- **backgroundColor: 'blue'**: Makes background blue.
- In JSX, CSS property names use camelCase. Example: background-color becomes backgroundColor.

You can also store styles in a variable:

```
const styleObj = {
color: "red",
backgroundColor: "blue"
};

function Home() {
return <h1 style={styleObj}>Hello World</h1>;
}
```

Method 2: External CSS Stylesheet

Create a separate CSS file and import it into the React component.

Style.css

```
.myClass {
color: red;
background-color: blue;
}
```

Home.js

```
import "./Style.css";

function Home() {
return (
<div className="myClass">
Hello World
</div>
);
}

export default Home;
```

Explanation

- import './Style.css' loads the CSS file.
- className='myClass' connects the div to the CSS class.
- In React, use className, not class.

3.1 CSS Modules

Simple Definition

A CSS Module is a CSS file with the extension .module.css. It creates unique class names automatically, so the same class name can be used in different components without conflicts.

Example

mystyle.module.css

```
.myClass {
color: red;
background-color: blue;
}
```

Home.js

```
import styles from "./mystyle.module.css";

function Home() {
return (
<div className={styles.myClass}>
Hello World
</div>
);
}

export default Home;
```

Explanation

- import styles from './mystyle.module.css' imports the CSS module as an object.
- styles.myClass accesses the myClass style.
- className={styles.myClass} applies the module style.

The PPT example uses both a normal CSS file and a CSS module. The CSS module style is applied, so the text becomes red with a blue background.

Comparison

Feature	Inline CSS	CSS Stylesheet	CSS Module
File extension	No separate file	.css	.module.css
Syntax	JavaScript object	Normal CSS	Normal CSS
Class name conflicts	Not applicable	Possible	Avoided
Best for	Very small styles	Global styles	Component-specific styles
Import required	No	Yes	Yes

Important Exam Points

- Inline CSS value must be a JavaScript object.
- Use double curly brackets: style={{}}.
- Use className in React, not class.
- CSS Modules use .module.css extension.
- CSS Modules prevent naming clashes.

4. React Hooks

Simple Definition

Hooks are special React functions that allow function components to use React features such as state and lifecycle methods.

Detailed Explanation

Before Hooks, state and lifecycle features were mainly used in class components. Hooks were introduced in React 16.8. Now, function components can use state and other React features without writing a class.

The word hook means that we connect or hook into React features.

Rules of Hooks

- Hooks can be called only inside React function components.
- Hooks must be called at the top level of a component.
- Hooks cannot be called inside conditions such as if statements.
- Hooks cannot be called inside loops or nested functions.

Incorrect Example

```
if (isLoggedIn) {
const [name, setName] = useState("");
}
```

Correct Example

```
const [name, setName] = useState("");

if (isLoggedIn) {
console.log(name);
}
```

Important Built-in Hooks

Hook	Purpose
useState	Stores and updates changing data
useEffect	Performs side effects such as API calls and timers
useContext	Shares data between components without props
useRef	Accesses DOM elements or stores mutable values
useReducer	Manages complex state logic
useCallback	Memoizes functions
useMemo	Memoizes calculated values

Important Exam Points

- Hooks were introduced in React 16.8.
- Hooks allow function components to use state and lifecycle features.
- Hooks must be imported from react.
- Hooks cannot be conditional.

5. useState Hook

Simple Definition

useState is a React Hook used to add and manage state in a function component.

What is State?

State means data that can change while the application is running.

Examples:
• Counter value.
• Username typed in a form.
• Login status.
• List of products.
• Selected menu item.

Syntax

```
const [stateVariable, setStateVariable] = useState(initialValue);
```

Example:

```
const [count, setCount] = useState(0);
```

Counter Example

```
import { useState } from "react";

function Counter() {
const [count, setCount] = useState(0);

return (
<>
<h1>{count}</h1>

<button onClick={() => setCount(count + 1)}>
Increase
</button>
</>
);
}

export default Counter;
```

Line-by-Line Explanation

Code	Meaning
import { useState } from 'react'	Imports useState Hook
const [count, setCount]	Destructures returned values from useState
useState(0)	Initial value of count is 0
count	Current state value
setCount	Function used to update count
{count}	Displays current count
setCount(count + 1)	Increases count by 1

How State Changes

```
Initial render:
count = 0
```

```
User clicks Increase button:
setCount(count + 1)

React updates state:
count = 1

React re-renders component:
<h1>1</h1>
```

Important Rule

Always update state using the setter function.

```
setCount(20); // Correct

count = 20; // Incorrect
```

Important Exam Points

- useState returns two values: current state and updater function.
- useState accepts an initial value.
- State must be updated using the setter function.
- When state changes, React re-renders the component.

6. useEffect Hook

Simple Definition

useEffect is a React Hook used to perform side effects in function components.

What is a Side Effect?

A side effect is any action that happens outside the normal rendering of a component.

Examples:
• Fetching data from an API.
• Updating the browser document title.
• Starting or stopping timers.
• Directly updating the DOM.
• Adding event listeners.

Syntax

```
useEffect(function, dependencyArray);
```

The second argument, the dependency array, is optional.

Example 1: Run After Every Render

```
useEffect(() => {
console.log("Component rendered");
});
```

Because no dependency array is provided, this effect runs after every render.

Example 2: Run Only Once

```
useEffect(() => {
console.log("Component loaded");
}, []);
```

An empty array [] means the effect runs only after the first render.

Example 3: Run When a Value Changes

```
useEffect(() => {
console.log("Count changed");
}, [count]);
```

This effect runs when the value of count changes.

Dependency Table

Code	When it runs
useEffect(() => {})	After every render
useEffect(() => {}, [])	Only once after first render
useEffect(() => {}, [count])	When count changes
useEffect(() => {}, [a, b])	When a or b changes

Important Exam Points

- useEffect performs side effects.
- It accepts two arguments: function and optional dependency array.

- Empty dependency array means run once.
- State or props can be used as dependencies.

7. Forms in React

Simple Definition

A form is used to collect input from the user, such as name, email, password, or other details.

HTML Form Behaviour

In plain HTML, a form can automatically send data and navigate to another page when submitted.

```
<form>
<label>
Name:
<input type="text" name="name" />
</label>

<input type="submit" value="Submit" />
</form>
```

In React, we usually want JavaScript to handle the submitted data. Therefore, we use controlled components.

Controlled Components

Simple Definition

A controlled component is a form input whose value is controlled by React state.

Detailed Explanation

In HTML, form elements such as input, textarea, and select keep their own internal state. In React, we make React state the single source of truth.

This means:
• React state stores the input value.
• The input displays that state value.
• When the user types, React updates the state.
• React re-renders the input with the new value.

Controlled Form Example

```
import { useState } from "react";

function MyForm() {
const [name, setName] = useState("");

function handleSubmit(event) {
event.preventDefault();
console.log(name);
}

return (
<form onSubmit={handleSubmit}>
<input
type="text"
value={name}
onChange={(event) => {
setName(event.target.value);
}}
/>

<input type="submit" value="Submit" />
</form>
);
}
```

```
export default MyForm;
```

Line-by-Line Explanation

Code	Meaning
const [name, setName] = useState('')	Creates empty state for name
value={name}	Input value comes from React state
onChange={...}	Runs when user types
event.target.value	Gets current typed value
setName(event.target.value)	Updates name state
onSubmit={handleSubmit}	Calls function when form is submitted
event.preventDefault()	Stops browser page refresh/navigation
console.log(name)	Displays entered name in console

8. Handling Multiple Inputs

Simple Definition

When a form has more than one input field, we can store all values in one state object.

Example: Username and Password

```
import { useState } from "react";

function MyForm() {
const [data, setData] = useState({
userName: "",
password: ""
});

function handleSubmit(event) {
event.preventDefault();

console.log(data.userName);
console.log(data.password);
}

return (
<form onSubmit={handleSubmit}>
<input
type="text"
value={data.userName}
onChange={(event) => {
setData({
...data,
userName: event.target.value
});
}}
/>

<input
type="password"
value={data.password}
onChange={(event) => {
setData({
...data,
password: event.target.value
});
}}
/>

<input type="submit" value="Login" />
</form>
);
}

export default MyForm;
```

Important Line

```
setData({
...data,
userName: event.target.value
});
```

Explanation:

• ...data copies all old values.
• userName: event.target.value updates only the username.
• Without ...data, the password value may be lost.

Important Exam Points

- Use one state object for multiple form inputs.
- Use spread operator ...data to preserve other values.
- Controlled input uses both value and onChange.

9. Consuming API in ReactJS

Simple Definition

Consuming an API means getting data from a server and displaying it in a React application.

What is an API?

API means Application Programming Interface. It allows two applications to communicate with each other.

Example flow:

```
React Application
|
| fetch() request
v
API Server
|
| sends JSON data
v
React Application displays data
```

What is JSON?

JSON means JavaScript Object Notation. It is a common format used to send data between a server and a web application.

```
[
{
"id": "1",
"FacultyName": "Rahul Patel",
"FacultyDepartment": "Computer Science",
"FacultySalary": 50000
}
]
```

Get All Records Example

```
import { useEffect, useState } from "react";

function Faculties() {
const [faculties, setFaculties] = useState([]);

useEffect(() => {
const apiUrl =
"https://your-project.mockapi.io/api/v1/faculties";

fetch(apiUrl)
.then((response) => response.json())
.then((result) => {
setFaculties(result);
});
}, []);

return (
<div>
{faculties.map((faculty) => (
<div key={faculty.id}>
<h3>{faculty.FacultyName}</h3>
<p>{faculty.FacultyDepartment}</p>
</div>
))}
</div>
);
}
```

```
export default Faculties;
```

Line-by-Line Explanation

Code	Meaning
const [faculties, setFaculties] = useState([])	Creates empty array state
useEffect(() => { ... }, [])	Runs API request once after component loads
fetch(apiUrl)	Sends GET request to API
response.json()	Converts API response into JavaScript data
setFaculties(result)	Stores API data in state
faculties.map(...)	Loops through every faculty record
key={faculty.id}	Gives each list item a unique key
faculty.FacultyName	Displays faculty name

10. Mock API using mockapi.io

Simple Definition

A mock API is a temporary fake API used for learning, testing, and development.

Steps to Create Mock API

- Go to mockapi.io.
- Sign up using email or Google, or log in.
- Click New Project.
- Enter project name and click Create Project.
- Click Add Resource.
- Enter resource name, such as users or faculties.
- Add fields such as name, age, email, salary, etc.
- Set field type: string, number, boolean, date, etc.
- Click Save.
- Click Generate to create sample records.
- Copy the API endpoint URL.

Example

Create a project named MyFirstProject. Add a resource named users. Add fields: name (string), age (number), email (string). Generate 10 users.

API Endpoint Format

```
https://<your-project-name>.mockapi.io/api/v1/<resource-name>
```

Example:

```
https://myfirstproject.mockapi.io/api/v1/users
```

Important Exam Points

- A mock API is used for development and testing.
- Each resource has a unique endpoint URL.
- CRUD means Create, Read, Update, Delete.
- HTTP methods: GET, POST, PUT, DELETE.

11. CRUD Operations using React

What is CRUD?

CRUD Operation	Meaning	HTTP Method	Example
Create	Add new record	POST	Add new faculty
Read	Get records	GET	Display all faculties
Update	Modify existing record	PUT	Edit faculty details
Delete	Remove record	DELETE	Delete a faculty

Five API Operations in PPT

- getAll: Get all faculty records.
- getByID: Get one faculty record using ID.
- insert: Add a new faculty record.
- update: Edit an existing faculty record.
- delete: Remove a faculty record.

Faculty API Fields

- FacultyID
- FacultyName
- FacultyImage
- FacultyDepartment
- FacultySalary

Get One Record by ID

```
import { useParams } from "react-router-dom";
import { useEffect, useState } from "react";

function FacultyDetail() {
const { id } = useParams();
const [faculty, setFaculty] = useState({});

useEffect(() => {
const apiUrl =
"https://your-project.mockapi.io/api/v1/faculties";

fetch(`${apiUrl}/${id}`)
.then((response) => response.json())
.then((result) => {
setFaculty(result);
});
}, [id]);

return (
<div>
<h1>{faculty.FacultyName}</h1>
<p>{faculty.FacultyDepartment}</p>
<p>{faculty.FacultySalary}</p>
<img src={faculty.FacultyImage} alt="Faculty" />
</div>
);
}
```

```
export default FacultyDetail;
```

Important Lines

- const { id } = useParams() reads ID from URL.
- fetch(`${apiUrl}/${id}`) requests one record.
- [id] dependency runs effect when ID changes.

Delete Record

```
function deleteFaculty(id) {
fetch(`${apiUrl}/${id}`, {
method: "DELETE"
})
.then(() => {
navigate("/faculties");
});
}
```

Explanation:

• method: 'DELETE' tells API to delete record.
• navigate('/faculties') returns to faculty list.

11.1 Insert and Update Records

Insert Record using POST

```
function addFaculty() {
fetch(apiUrl, {
method: "POST",
body: JSON.stringify(data),
headers: {
"Content-Type": "application/json"
}
})
.then(() => {
navigate("/faculties");
});
}
```

Explanation

- method: 'POST' creates a new record.
- JSON.stringify(data) converts JavaScript object into JSON text.
- Content-Type: application/json tells server that request body is JSON.
- navigate('/faculties') returns to list page after adding.

Update Record using PUT

```
function updateFaculty(id, data) {
fetch(`${apiUrl}/${id}`, {
method: "PUT",
body: JSON.stringify(data),
headers: {
"Content-Type": "application/json"
}
})
.then(() => {
navigate("/faculties");
});
}
```

POST vs PUT

Method	Purpose	URL includes ID?
POST	Create new record	No
PUT	Update existing record	Yes

Dynamic Route

```
<Route
path="/faculties/:id"
element={<FacultyDetail />}
/>
```

:id is a dynamic route parameter. For URL /faculties/5, useParams() returns id = 5.

12. Exam Practice Questions

Short Answer Questions

Question	Answer
What is routing in React?	Routing means displaying different components based on the browser URL.
Which package is used for routing in React?	react-router-dom
What is the use of BrowserRouter?	It enables routing in a React application.
What is the use of Outlet?	It displays the child route component inside a parent layout.
What is a Hook?	A Hook is a special React function that allows function components to use React features.
What is useState?	useState is a Hook used to store and update changing data in a function component.
What is useEffect?	useEffect is a Hook used to perform side effects such as API calls and timers.
What is a controlled component?	A form input whose value is controlled by React state.
What is an API?	API means Application Programming Interface. It allows applications to communicate.
What is CRUD?	CRUD means Create, Read, Update, and Delete.

MCQs

Question	Options	Correct Answer	Explanation
Which package is installed for React routing?	A. react-router B. react-router-dom C. router-dom D. react-route	B	react-router-dom provides routing components for web applications.
Which component enables routing?	A. Routes B. Route C. BrowserRouter D. Link	C	BrowserRouter enables routing.
Which component displays child routes?	A. Outlet B. Link C. Route D. Navigate	A	Outlet displays the matched child route.
Which Hook stores changing data?	A. useEffect B. useState C. useRef D. useMemo	B	useState stores and updates state.
Which Hook performs side effects?	A. useState B. useEffect C. useContext D. useReducer	B	useEffect handles side effects.
Which HTTP method creates a new record?	A. GET B. PUT C. POST D. DELETE	C	POST creates a new record.
Which HTTP method updates a record?	A. GET B. POST C. PUT D. DELETE	C	PUT updates an existing record.
Which HTTP method deletes a record?	A. GET B. POST C. PUT D. DELETE	D	DELETE removes a record.
In React, which attribute is used instead of class?	A. class B. classname C. className D. css	C	React uses className.
Which file extension is used for CSS Modules?	A. .css B. .module.css C. .scss D. .style	B	CSS Modules use .module.css.

13. Long Answer Questions

Q1. Explain Routing in React with Example

Routing means displaying different components based on the URL. React does not include routing by default, so we install react-router-dom.

Important components:
• BrowserRouter: Enables routing.
• Routes: Contains route definitions.
• Route: Connects URL to component.
• Link: Creates navigation link.
• Outlet: Displays child route.

```
root.render(
<BrowserRouter>
<Routes>
<Route path="/" element={<Layout />}>
<Route index element={<Home />} />
<Route path="about" element={<About />} />
<Route path="contact" element={<Contact />} />
</Route>
</Routes>
</BrowserRouter>
);
```

When the user opens /about, React displays the Layout component and the About component inside Outlet.

Q2. Explain useState Hook with Example

useState is used to add state to a function component. State is data that can change during application execution.

Syntax:

```
const [count, setCount] = useState(0);
```

useState returns two values:
• count: Current state.
• setCount: Function to update state.

```
function Counter() {
const [count, setCount] = useState(0);

return (
<>
<h1>{count}</h1>
<button onClick={() => setCount(count + 1)}>
Increase
</button>
</>
);
}
```

Initially, count is 0. When the button is clicked, setCount(count + 1) changes count to 1, and React re-renders the component.

Q3. Explain useEffect Hook with Example

useEffect performs side effects in React function components. Side effects include fetching data, updating DOM, and using timers.

Syntax:

```
useEffect(function, dependencyArray);
```

There are three common dependency cases:

- No dependency: Runs after every render.
- Empty array []: Runs only once.
- Specific dependency [count]: Runs when count changes.

```
useEffect(() => {
fetch(apiUrl)
.then((response) => response.json())
.then((result) => setData(result));
}, []);
```

This example fetches API data once after the component loads.

14. Final Revision

Important Definitions

Term	Definition
Routing	Displaying different components based on URL
Component	Reusable part of a React UI
Hook	Special React function for using React features in function components
State	Data that can change during application execution
Controlled Component	Form input controlled by React state
API	Interface that allows applications to communicate
JSON	JavaScript Object Notation; common data format for APIs
Endpoint	URL used to access an API resource
CRUD	Create, Read, Update, Delete operations

Important Syntax

```
npm install react-router-dom

const [state, setState] = useState(initialValue);

useEffect(() => {
// side effect
}, [dependency]);

fetch(apiUrl)
.then((response) => response.json())
.then((result) => setData(result));

fetch(apiUrl, {
method: "POST",
body: JSON.stringify(data),
headers: {
"Content-Type": "application/json"
}
});
```

Important Comparisons

Concept	Difference
BrowserRouter vs Routes	BrowserRouter enables routing; Routes contains route definitions
Route vs Link	Route connects URL to component; Link creates navigation link
Outlet vs Route	Route defines a URL; Outlet displays child route component
useState vs useEffect	useState stores data; useEffect performs side effects
GET vs POST	GET reads data; POST creates data
POST vs PUT	POST creates a new record; PUT updates an existing record
Inline CSS vs CSS Module	Inline CSS is written in JSX; CSS Module is a separate .module.css file

Last-Minute Checklist

- Routing package: react-router-dom.
- BrowserRouter enables routing.
- Routes contains Route components.
- Link navigates without page reload.
- Outlet displays child route.
- useState returns state and setter function.
- Never write count = 20; use setCount(20).
- useEffect with [] runs once.
- Controlled input uses value and onChange.
- event.preventDefault() stops page refresh.
- fetch() communicates with API.
- response.json() converts response to JavaScript data.
- map() displays list data.
- key={item.id} is required in lists.
- useParams reads URL parameter.
- useNavigate changes route programmatically.
- GET reads, POST creates, PUT updates, DELETE removes.